006-Glang

Difficulty: Hard Expect Time: 60-90 min Author: Yan Hau, Chen

Gamelab aims to develop a dynamic scripting language called **Glang**, designed for embedding within games. Similar to languages like *Python* or *JavaScript*, **Glang** allows variables to hold different types of values—such as integers, strings, or booleans—and to change types dynamically at runtime.

- **Type System**

  The **`var`** in **Glang** must be able to represent the following scalar types:

  - Int - 64-bit signed integers (**`int`**)
  - Float - Floating-point numbers (**`float`** and **`double`**)
  - Boolean - Boolean values **`true`** and **`false`**
  - String - Text values, using **`std::string`**. Implicitly constructed from string literals

  NOT Required to support function objects (e.g., lambda), pointers, arrays, or any kind of composite or reference-based types.

  The types **`int`**, **`float`**, and **`double`** are all treated as **`numeric types`** in **Glang** and can be freely combined in **arithmetic operations** through implicit conversion to double.

- Operators and Type Coercion

  Glang supports the following **binary operations** with permissive type coercion semantics:

  - **`operator+()`**
    Try concatenating two var instances containing strings, or performing addition between two var values.
  - **`operator-()`**
    Try subtracting one var value from another.
  - **`operator*()`**
    Try multiplying two var values.
  - **`operator/()`**
    Try dividing one var value by another.

- String & String : String concatenation

  a) **`var("1234") + var("1234") == var("12341234")`**

  b) **`var("ABC") + "789"s == var("ABC789")`**

- Numeric & Numeric : convert to double for calculate
  a) `var(1) + 2.5 == var(3.5)`
  b) `var(3) - 1.0f == var(2.0)`
  c) `var(4.0) * 2 == var(8.0)`
  d) `var(10) / 2 == var(5.0)`

- Numeric & String : Attempt to convert the string to a double before performing the calculate.
  a) `var(1) + var("2.5") == var(3.5)`
  b) `var("4.2") * var(2) == var(8.4)`
  c) `var("3.0") - var("1.5") == var(1.5)`
  d) `var("3.0") / var(1.5) == var(2.0)`

➢ Error Handing

In this problem, the following three types of exceptions are defined:

- **`std::invalid_argument("incompatible types")`**
  When a **`bool type`** participates in an arithmetic operation

- **`std::invalid_argument("convert failed")`**
  Thrown when a string fails to convert to a numeric value during implicit coercion.

- **`std::domain_error("not allow devide by 0")`**
  Thrown when division by zero occurs, including cases where zero is parsed from a string.

For more details about operator behavior and type coercion rules, please refer to the implementation in **`operator.h`**.

➢ Constructor and Copy Constructor

This problem does not provide a default constructor definition. Please implement the constructor so that values of type **`int`**, **`float`**, **`double`**, **`bool`**, and **`std::string`** can be correctly stored.

➢ Destructor

NOT required to implement a destructor for this problem. However, you may do so **if you believe it is necessary**.

In this problem, provided files: **`value.h`**, **`variable.h`** and **`operator.h`**. These serve as the foundational components of the system.

Your task is to implement the **`var class`**, which is **NOT a template class**. Instead, you are expected to combine **compile-time and runtime polymorphism** to achieve behavior. Specifically, the design should use **template-based type capture at compile time**, and **virtual table dispatch at runtime** through a base–derived class hierarchy. Understanding how the virtual table mechanism enables dynamic behavior is **essential to completing this task**.

Parts of the **`var class`** definition are provided in **`variable.h`**. To minimize potential confusion related to templates, the generic implementation of **`as<T>()`** has already been defined to avoid ambiguity during compilation. In addition to the given code, you are expected to determine the appropriate concrete type for **`var::Dispatcher`** and redeclare it accordingly.

If you find that the current design of the **`var class`** is insufficient to meet the problem requirements, **ALLOWED** to add more members, methods, constructors, or supporting structures as needed.

Input

1. Complete the required class implementation. The following files already contain some implementations
   a) **`variable.h`**
   b) **`value.h`**
   c) **`operator.h`**
   You can submit a header-only implementation, or split the implementation into **`*.cpp`** files.

2. The Online Judge system will replace the following files:
   a) **`entrypoint.cpp`**
   b) **`case.h`**
   c) **`test.h`**

3. The test cases for student available in **`case.h`**, but not all test data is included in that file. The Online Judge system will also include hidden test inputs.

4. When declaring **`float`**, **`double`**, or **`std::string`** values, the corresponding literal suffix (such as **`f`** for **`float`** or **`s`** for **`std::string`**) must be **explicitly** used.
   a) **`var a = 100`** will be treated as **`int`**
   b) **`var b = 100.`** will be treated as **`double`**
   c) **`var c = 100.f`** will be treated as **`float`**
   d) **`var d = "1234"s`** will be treated as **`std::string`**
   e) **`var e = true`** will be treated as **`bool`**

5. If encounter **`var(str) + var(str)`**, you should prioritize string concatenation over arithmetic addition.

6. Cases that match multiple error conditions simultaneously will not occur. For example, **`"string" / 0`** could be interpreted as both an **`invalid_argument`** and a **`domain_error`**, but such cases will **NOT** appear.

Output

Online Judge will run the test:

1. **DON' T** output any information to the terminal.
2. When any test failed, Online Judge will try to list error messages.